Mathematical Foundations of Reinforcement Learning and Online Decision Making

Lecture 10 - Beyond the Classic Bandit

Sleeping, restless and other bandit families

Venktesh Viswanathan

October 24, 2026

Overview

1. When the classic bandit breaks
2. Sleeping bandits
3. Restless bandits
4. More bandit families

The bandit we have used so far

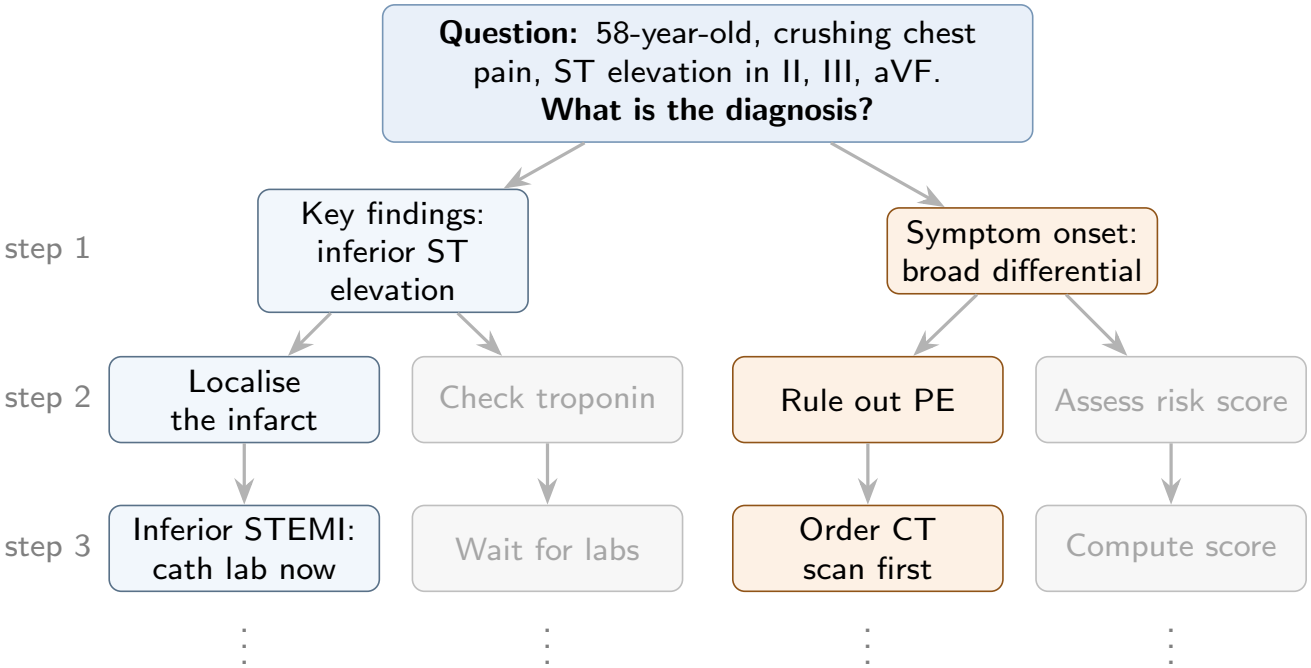
1. **Fixed Arm Space:** We saw settings where the arm space was fixed. The same arms are available in every round.
2. **Frozen arms:** an arm's reward distribution never changes.
3. **Scores as feedback:** the pulled arm returns a number.

The bandit we have used so far

1. **Fixed Arm Space:** We saw settings where the arm space was fixed. The same arms are available in every round.
2. **Frozen arms:** an arm's reward distribution never changes.
3. **Scores as feedback:** the pulled arm returns a number.

Hybrid Test-time scaling breaks *every one* of these. Each broken assumption gives a new family of bandits.

Back to Deep Thought



- **sleeping:** only the children of the current node can be expanded now.
- **restless:** unchecked branches keep growing and may drift off track.

Which assumption breaks? A map

In test-time scaling . . .	bandit family
tools / candidates only sometimes available	**sleeping**
chains evolve whether or not we check	**restless**
best strategy depends on the question	contextual
judge compares two answers, no score	dueling
we pick a *set* of k exemplars	combinatorial
judge or questions change over time	non-stationary / adversarial
verifier calls and tokens are budgeted	bandits with knapsacks

Today: the first two in depth.

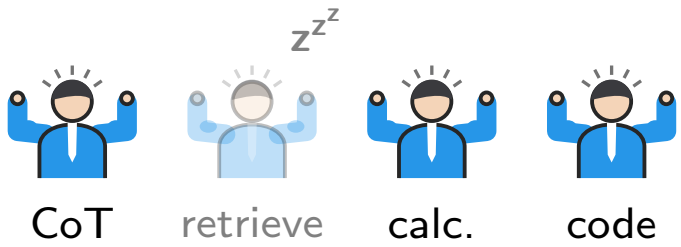
Not every arm is awake

Arms = ways to spend test-time compute. Which ones apply depends on the question.

What is the protocol to handle the virus causing acute respiratory syndrome?



Repealing the boat sales tax spawned 2,000 companies. How many existed before?



Calculator and code only help numeric questions; retrieval needs a trusted corpus for the domain.

Sleeping bandit: in each round only a subset of arms is available.

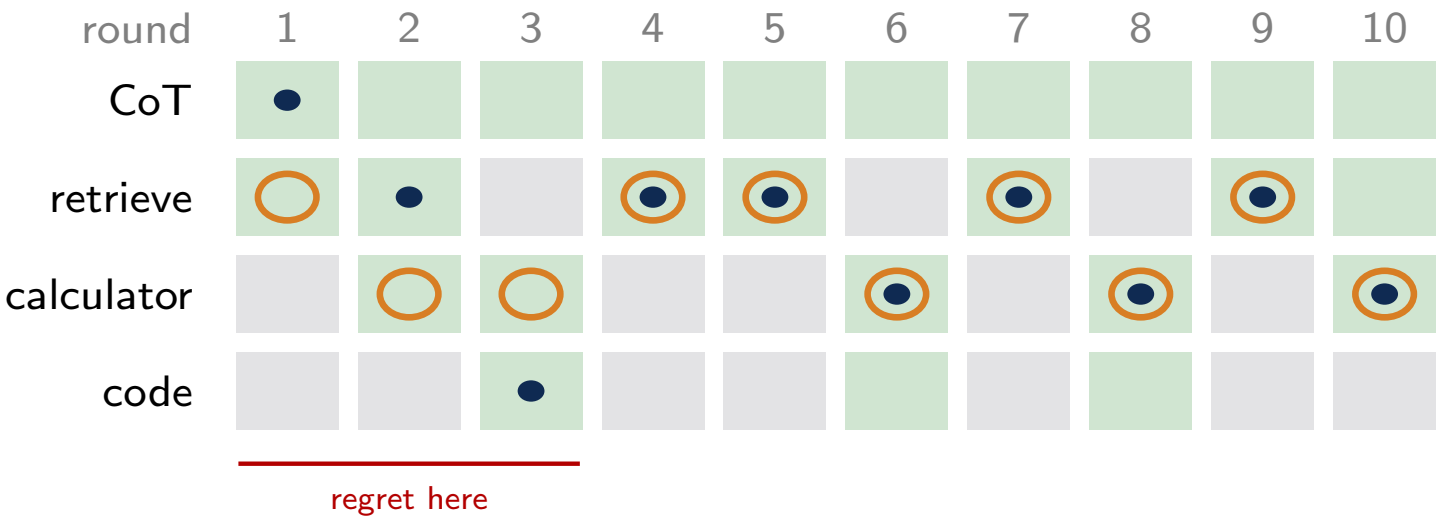
The sleeping bandit protocol

For rounds $t = 1, 2, \ldots, n$

1. The environment reveals the **awake set** $\mathcal{A}_t \subseteq [K]$.
2. The learner picks an awake arm $A_t \in \mathcal{A}_t$.
3. It observes the reward X_t of A_t only.

- Rewards: stochastic, each arm has a fixed mean μ_a.
- Availability: may be random or even chosen by an adversary.
- Key assumption: *whether* an arm is awake does not change its mean.

Who was awake when?



Green: awake, grey: asleep. Means: calculator 0.8 $>$ retrieve 0.7 $>$ code 0.65 $>$ CoT 0.55.

◯ best awake arm • learner's pick: they differ only while it is still exploring.

Regret against what?

Unfair benchmark: the best single arm. It may be asleep, so nobody could have played it.

Fair benchmark: the best *ordering* of the arms, "always play the highest-ranked arm that is awake". With stochastic rewards this is just "play the best awake arm":

$$R_n = \sum_{t=1}^{n} \left(\max_{a \in \mathcal{A}_t} \mu_a - \mu_{A_t} \right).$$

You are only compared with someone who had *the same options* as you.

AUER: UCB on the awake arms

Awake Upper Estimated Reward (Kleinberg et al., 2010)

$$A_t = \arg\max_{a \in \mathcal{A}_t} \left(\widehat{\mu}_a(t-1) + \sqrt{\frac{2 \log t}{T_a(t-1)}} \right)$$

- Exactly UCB, with the argmax restricted to the awake set.
- An arm's interval only shrinks when it is awake *and* pulled.

AUER: UCB on the awake arms

Awake Upper Estimated Reward (Kleinberg et al., 2010)

$$A_t = \arg\max_{a \in \mathcal{A}_t} \left(\widehat{\mu}_a(t-1) + \sqrt{\frac{2 \log t}{T_a(t-1)}} \right)$$

- Exactly UCB, with the argmax restricted to the awake set.
- An arm's interval only shrinks when it is awake *and* pulled.
- Logarithmic regret against the best ordering, and essentially optimal in the stochastic setting.

AUER: UCB on the awake arms

Awake Upper Estimated Reward (Kleinberg et al., 2010)

$$A_t = \arg\max_{a \in \mathcal{A}_t} \left(\widehat{\mu}_a(t-1) + \sqrt{\frac{2 \log t}{T_a(t-1)}} \right)$$

- Exactly UCB, with the argmax restricted to the awake set.
- An arm's interval only shrinks when it is awake *and* pulled.
- Logarithmic regret against the best ordering, and essentially optimal in the stochastic setting.

Optimism still works; it just has to respect who is awake.

Why sleeping is subtle

- **Rarely awake arms stay uncertain** for a long time, but they also rarely matter, so this costs little.
- **Availability must not depend on quality.** If the calculator is awake exactly on the hardest questions, its mean depends on *when* it is awake: that is a *contextual* bandit.
- **Adversarial availability and rewards**: no efficient algorithm is known to compete with the best ordering (Kanade & Steinke, 2014).

Sleeping arms with features come for free

Lecture 8's OFUL already allowed a **different action set every round**:

$$A_t = \arg\max_{a \in \mathcal{A}_t} \langle a, \widehat{\theta}_{t-1} \rangle + \beta_{t-1} \|a\|_{V_{t-1}^{-1}},$$

where the environment chooses $\mathcal{A}_t \subseteq \mathbb{R}^d$ freely each round.

- The confidence ellipsoid does not care which actions were offered.
- The same $\tilde{O}(d\sqrt{n})$ regret holds against the best awake action.

Example: exemplar selection where only exemplars that fit the remaining context window are awake. Shared features let you learn about an exemplar even while it sleeps.

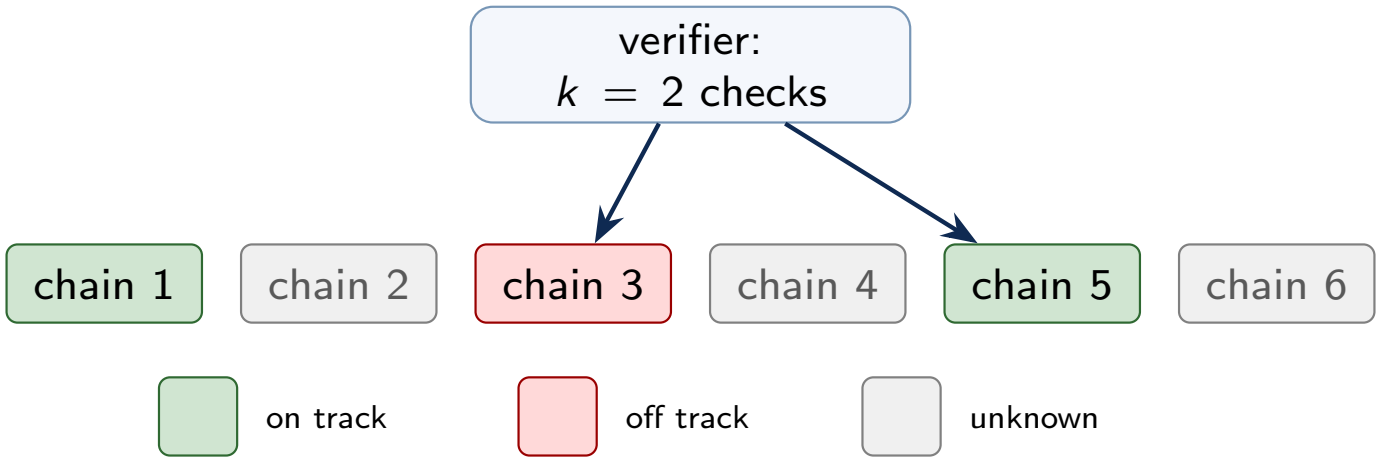
Sleeping bandits in LLM pipelines

- **Which verifier to call?** Unit tests are awake only for code, a calculator check only for numeric steps, an LLM judge always.
- **Which exemplars to show?** Only those from the question's domain, or that fit the context budget.
- **Which reasoning move next?** "Retrieve", "compute", "backtrack" are available only in some states of the search.
- **Which candidate paths to rank?** New paths appear and old ones are pruned as the search tree grows.

Whenever the menu changes but the dishes do not, think "sleeping".

Arms that change while you are not looking

Deep Thought runs N reasoning chains in parallel. Every step each chain writes more and may drift off track; we can **check only** k per step.

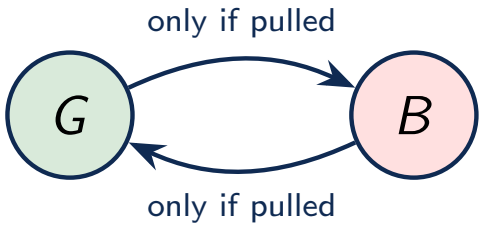


- Checking a chain reveals its state and lets us repair or prune it.
- Unchecked chains *keep evolving*: on-track chains can silently drift.

The arms move whether or not you pull them: a **restless** bandit.

Rested vs restless

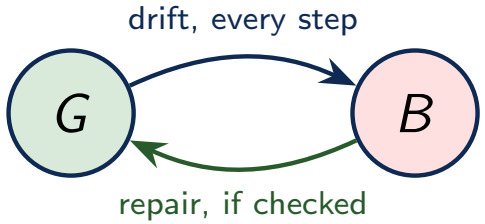
Rested (classic Markov bandit)



Not pulled ⇒ frozen.

Gittins index is optimal (Gittins, 1979).

Restless



Moves every round, with *passive* and *active* dynamics.

No Gittins-style optimal index.

Leaving an arm alone is itself a decision with consequences.

The restless bandit model (Whittle, 1988)

Restless multi-armed bandit

- N arms; arm i has a state $s_i(t)$ (a Markov chain).
- Each round: choose exactly k arms to be **active**.

The restless bandit model (Whittle, 1988)

Restless multi-armed bandit

- N arms; arm i has a state $s_i(t)$ (a Markov chain).
- Each round: choose exactly k arms to be **active**.
- Active arms: How states change when an arm is active is denoted by the transition probability matrix P^1, state changes in passive arms are rpresented by the transition probability matrix P^0.

The restless bandit model (Whittle, 1988)

Restless multi-armed bandit

- N arms; arm i has a state $s_i(t)$ (a Markov chain).
- Each round: choose exactly k arms to be **active**.
- Active arms: How states change when an arm is active is denoted by the transition probability matrix P^1, state changes in passive arms are rpresented by the transition probability matrix P^0.
- Every arm pays a reward $r(s_i(t))$, active or not.
- Goal: maximize the long-run total reward.

The restless bandit model (Whittle, 1988)

Restless multi-armed bandit

- N arms; arm i has a state $s_i(t)$ (a Markov chain).
- Each round: choose exactly k arms to be **active**.
- Active arms: How states change when an arm is active is denoted by the transition probability matrix P^1, state changes in passive arms are rpresented by the transition probability matrix P^0.
- Every arm pays a reward $r(s_i(t))$, active or not.
- Goal: maximize the long-run total reward.

- With known dynamics this is an MDP *planning* problem.
- With unknown dynamics we must also *learn* P^0, P^1: a bandit on top.

Running example: keep the chains on track

States: on track G / off track B.

Passive: $G \to B$ with drift rate p_i.

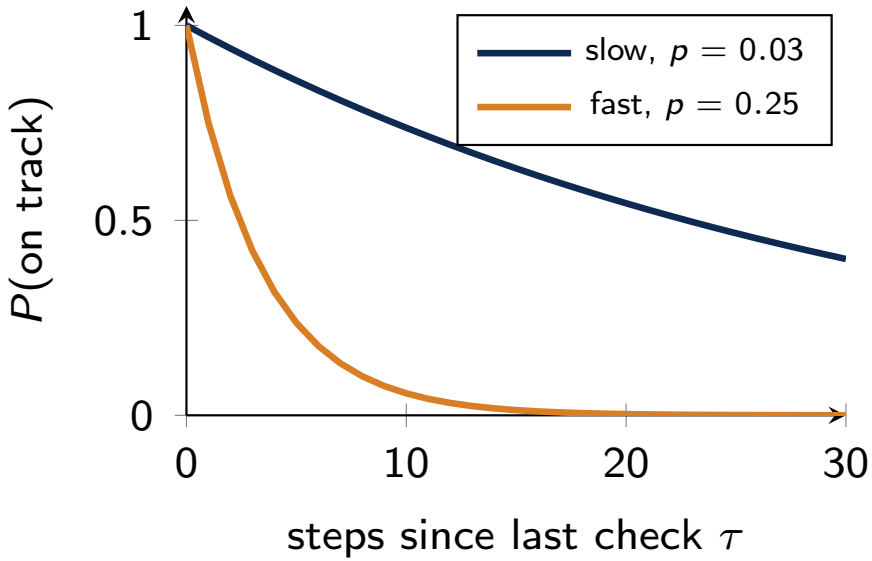
Active (check + repair): back to G.

Reward: 1 per on-track chain per step.

Unchecked chains are unseen, so we track the **belief**

$$b_i(\tau) = (1 - p_i)^{\tau},$$

τ = steps since the last check.



Which k chains should we check now?

Why not just solve the MDP? (1) State explosion

To plan, we need the state of *every* chain at once:

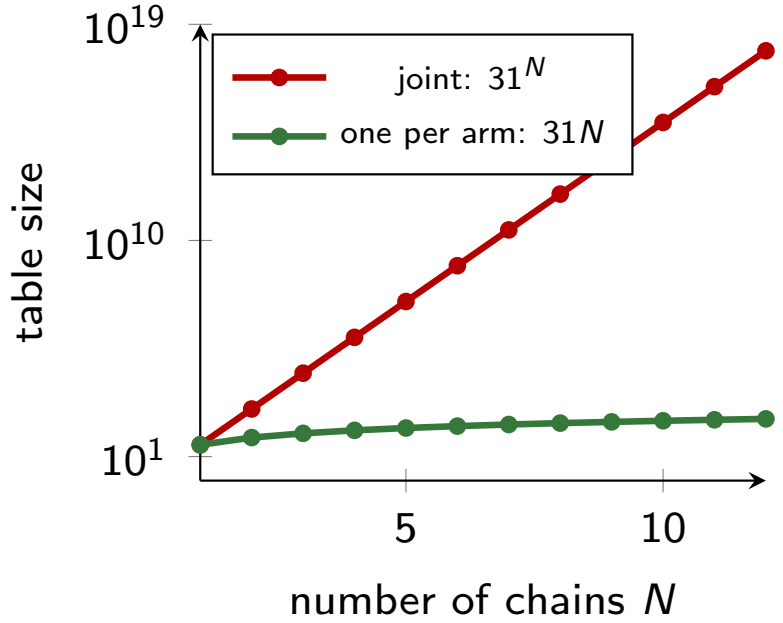
$$(\tau_1, \tau_2, \ldots, \tau_N),$$

τ_i = steps since chain i was checked.

With $\tau_i \in \{0, \ldots, 30\}$ and $N = 12$ chains:

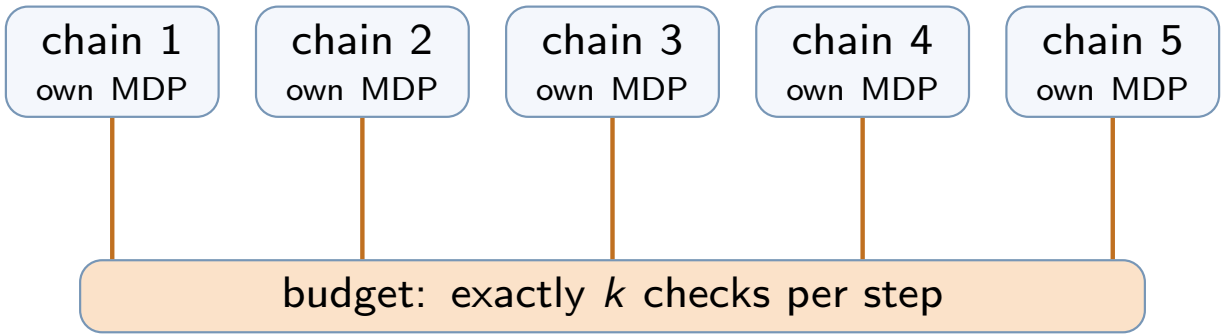
- joint states: $31^{12} \approx 8 \times 10^{17}$;
- actions per step: $\binom{12}{2} = 66$.

Value iteration needs a table entry for every joint state.

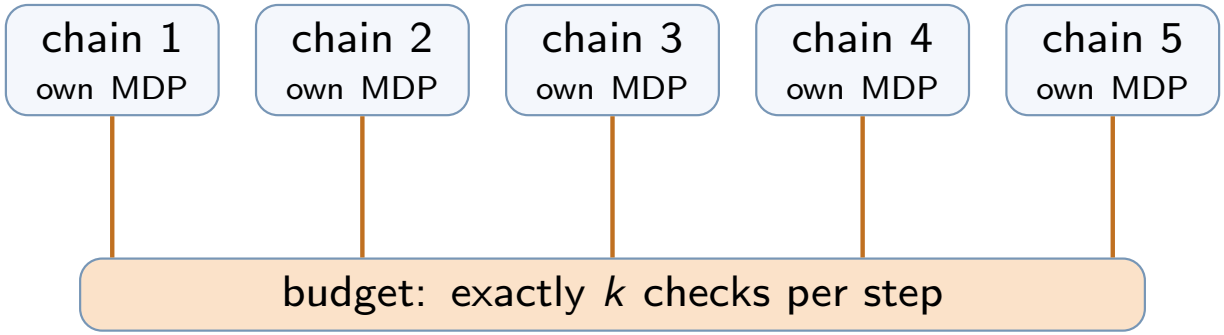


Every extra chain multiplies the table by 31. Planning per arm would only *add* 31 entries.

Why not just solve the MDP? (2) Coupling

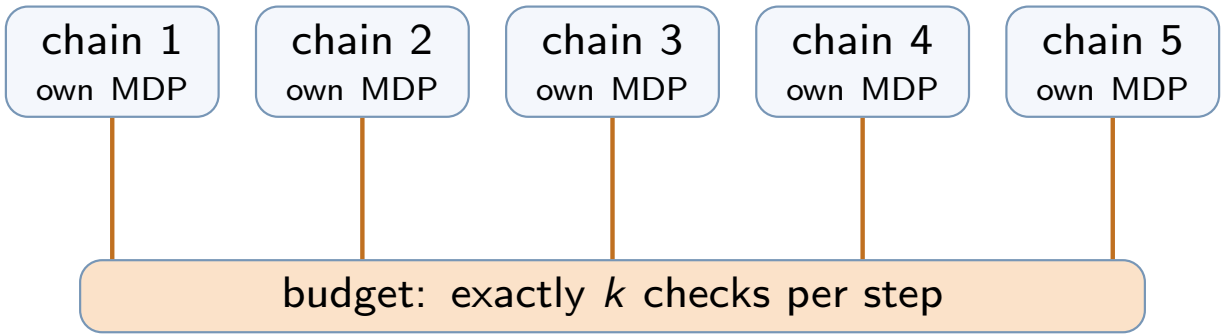


Why not just solve the MDP? (2) Coupling



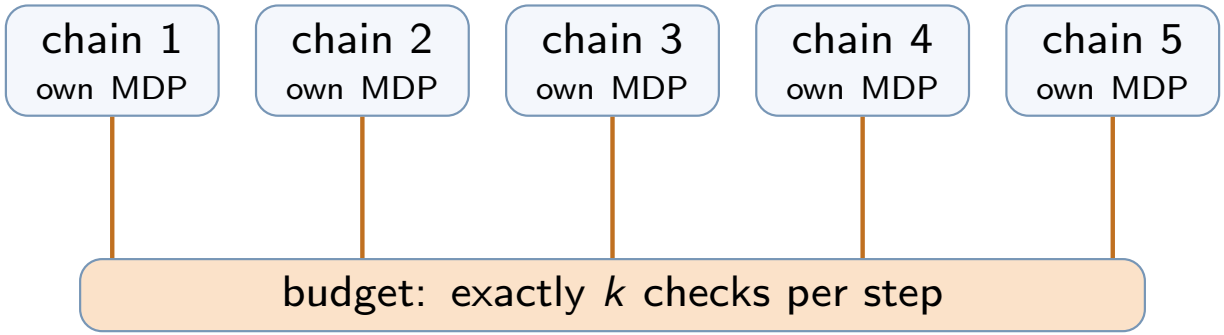
- **Without the budget**, each chain is a tiny MDP on its own: solve 12 problems of 31 states each.

Why not just solve the MDP? (2) Coupling



- **Without the budget**, each chain is a tiny MDP on its own: solve 12 problems of 31 states each.
- **With the budget**, checking chain 3 means *not* checking chain 5, and chain 5 keeps drifting meanwhile.

Why not just solve the MDP? (2) Coupling



- **Without the budget**, each chain is a tiny MDP on its own: solve 12 problems of 31 states each.
- **With the budget**, checking chain 3 means *not* checking chain 5, and chain 5 keeps drifting meanwhile.
- So the value of acting on one arm depends on the states of *all* the others: its **opportunity cost**.

One shared constraint turns N easy problems into one huge problem.

Why not just solve the MDP? (3) Waiting is not free

Rested arms (Gittins): an arm you skip is frozen. Skipping only *postpones* its rewards, so each arm can be scored on its own, and the Gittins index is exactly optimal.

Restless arms: an arm you skip keeps changing. Skipping chain 5 today makes it *worse* tomorrow, so the order of checks matters for everyone. The "score each arm alone" argument breaks.

Why not just solve the MDP? (3) Waiting is not free

Rested arms (Gittins): an arm you skip is frozen. Skipping only *postpones* its rewards, so each arm can be scored on its own, and the Gittins index is exactly optimal.

Restless arms: an arm you skip keeps changing. Skipping chain 5 today makes it *worse* tomorrow, so the order of checks matters for everyone. The "score each arm alone" argument breaks.

How hard is it really?

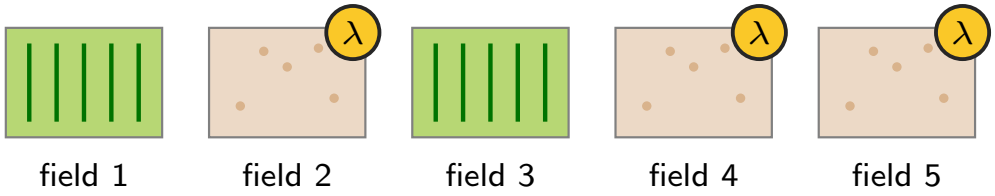
Finding the optimal policy is PSPACE-hard, even with known dynamics (Papadimitriou & Tsitsiklis, 1999). Not just a big table: no efficient method is expected at all. With *unknown* dynamics, we must also learn.

We need to break the coupling and think about one arm at a time: that is exactly what Whittle does next.

An analogy: paying fields to rest



The government wants only $k = 2$ fields farmed this year. How should it decide which ones?



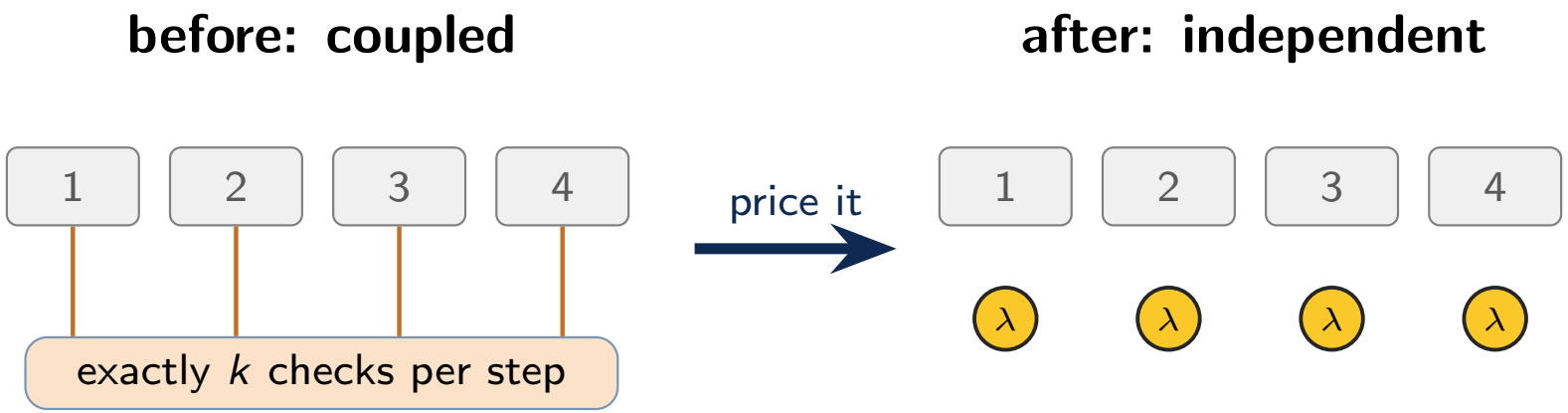
Plan centrally: needs every field's soil and history at once $\Rightarrow$ the joint MDP

Pay λ per fallow field: each farmer decides alone; raise λ until only k farm $\Rightarrow$ Whittle

Real programmes pay farmers to rest land: e.g. EU set-aside, the US Conservation Reserve Program.

field $\leftrightarrow$ chain farm $\leftrightarrow$ check fallow $\leftrightarrow$ rest fallow subsidy $\leftrightarrow \lambda$

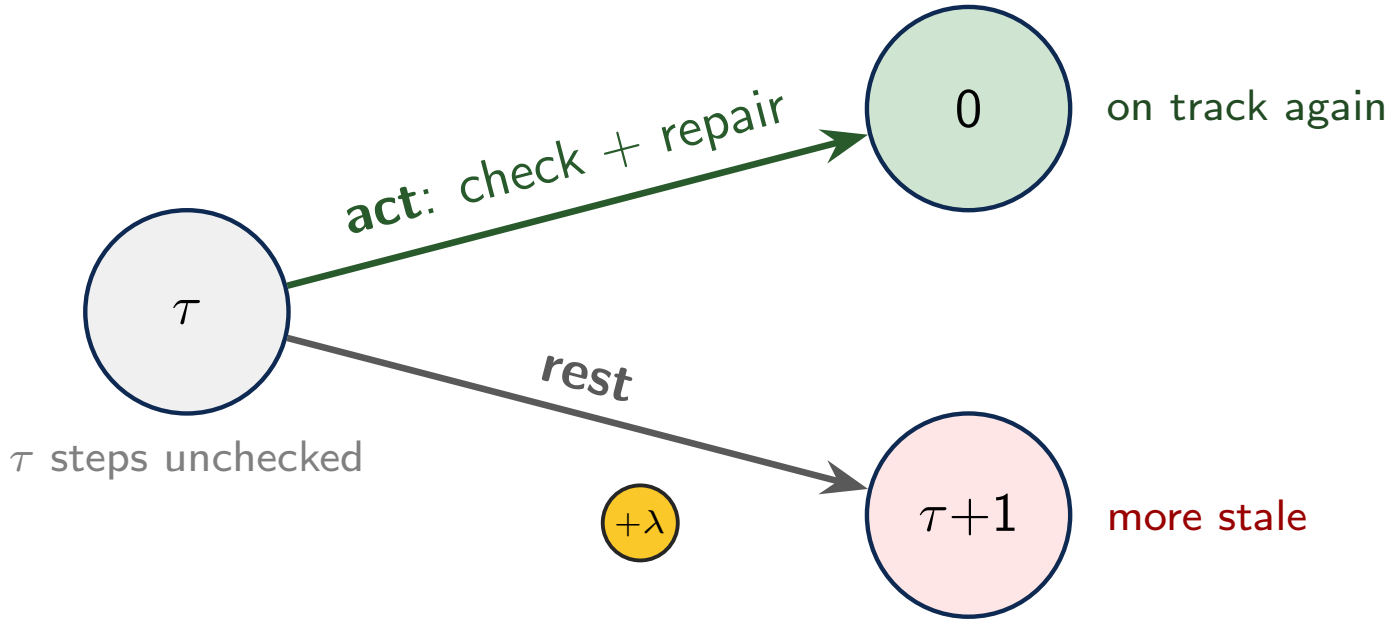
Whittle's idea: price the budget



coin λ = a fallow subsidy, paid each step a chain *rests*

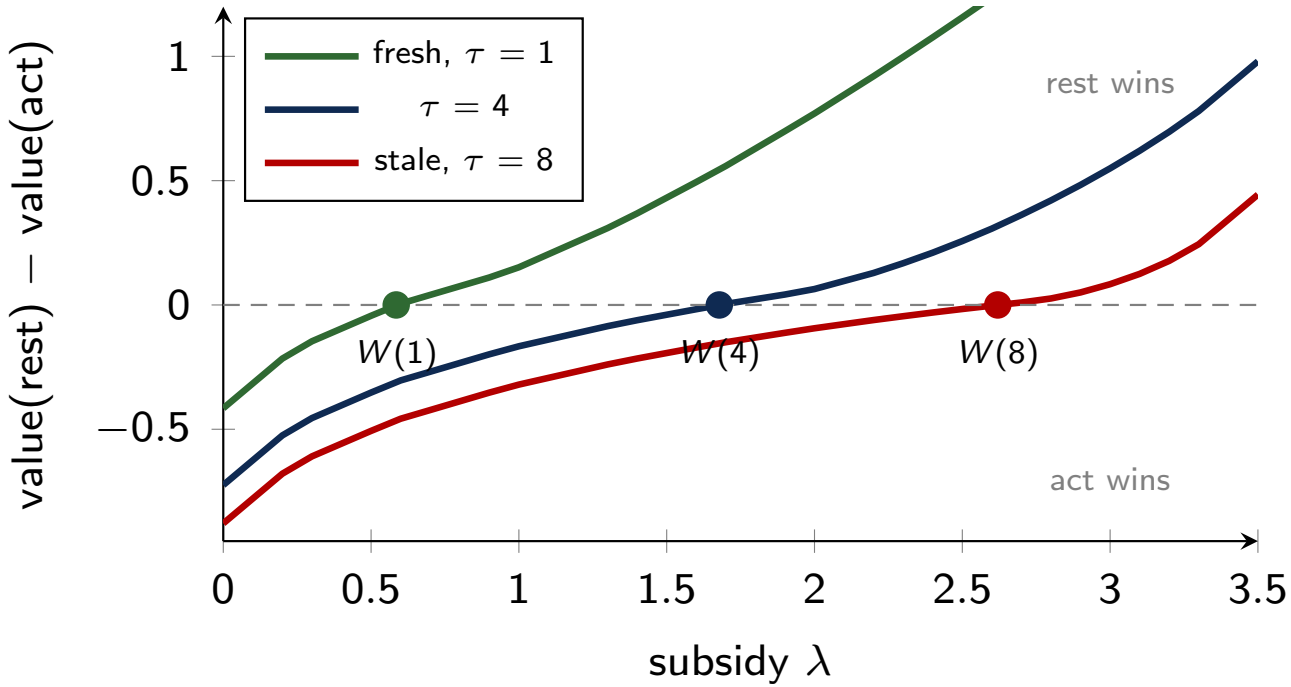
One coupled problem with N chains $\Rightarrow$ N separate one-chain problems.

One chain alone: act or rest?



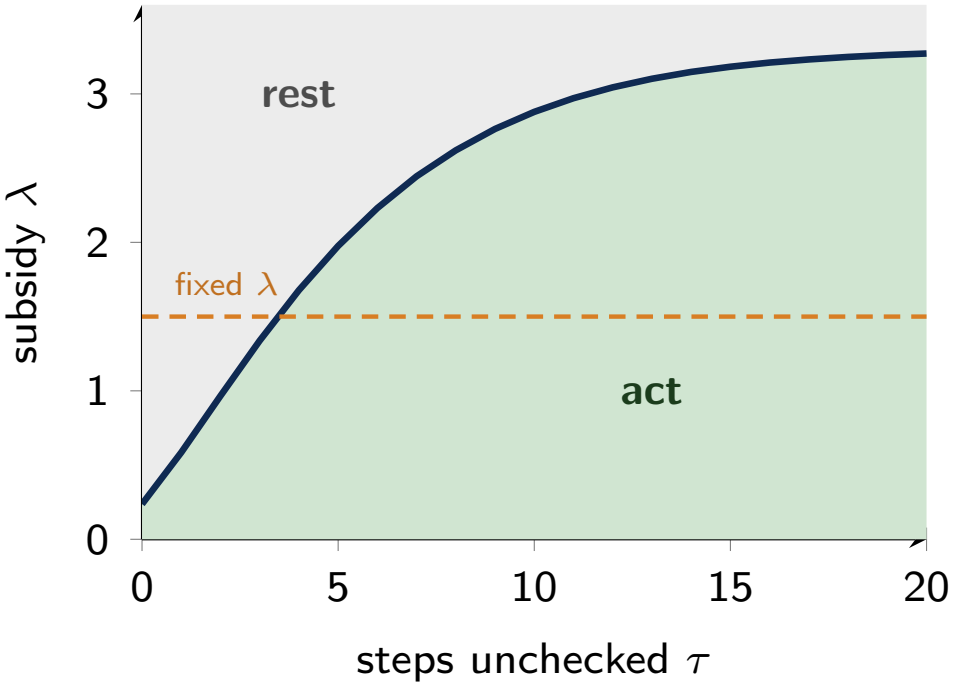
- Small λ or a stale chain $\Rightarrow$ **act**.
- Large λ or a freshly checked chain $\Rightarrow$ **rest**.

The Whittle index: the break-even subsidy



$W(\tau)$ = the subsidy at which acting and resting are equally good.
Stale chains need a bigger bribe to be left alone.

Act or rest, for every state



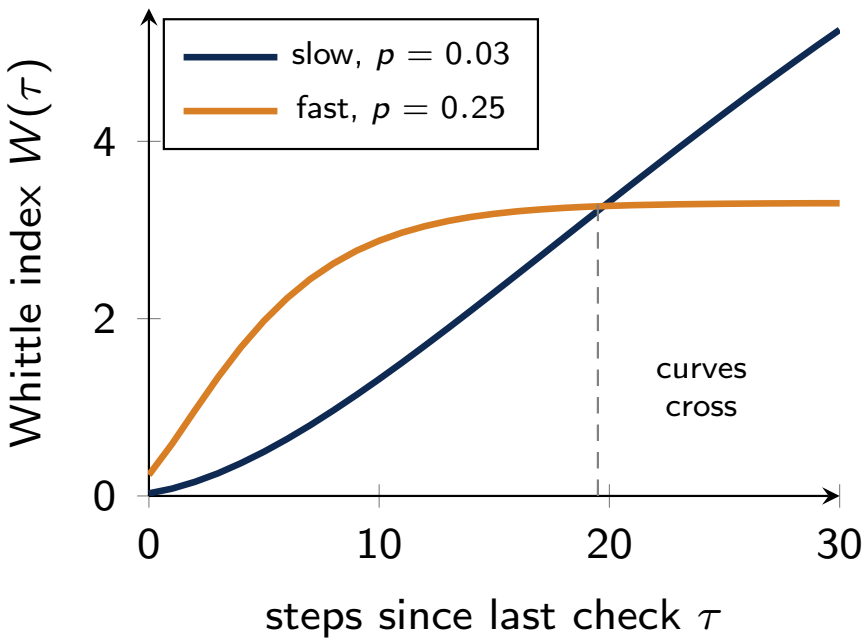
Fast chain ($p = 0.25$).

Boundary curve = the Whittle index $W(\tau)$.

At a fixed λ (dashed): stale states act, fresh states rest.

Indexable: raising λ only moves states from act to rest.

The Whittle index in the running example

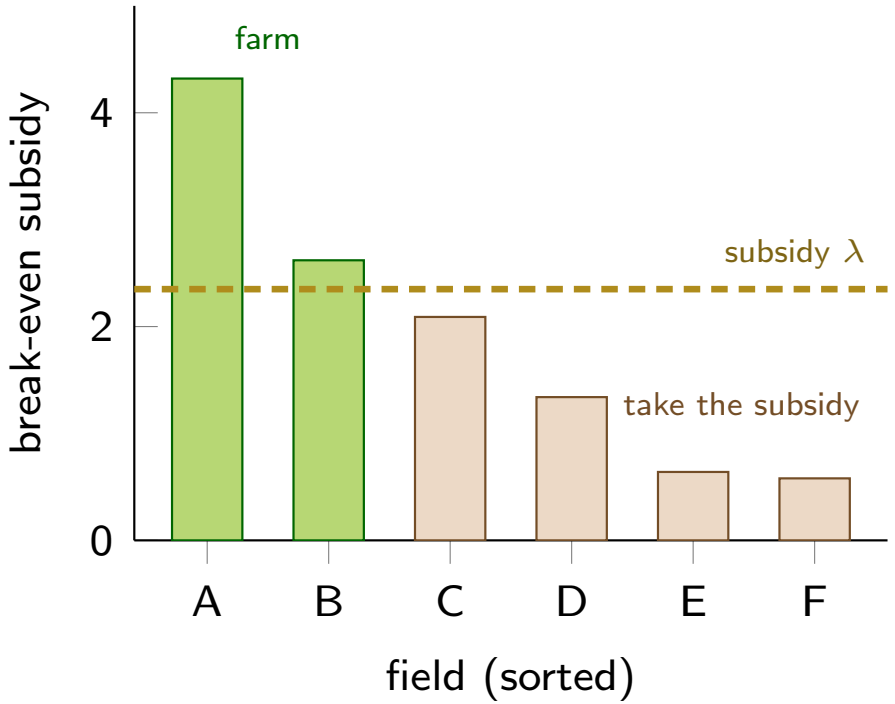


Computed numerically (discount 0.95).

- Index grows with time since the last check.
- Early on, **fast drifters are urgent**.
- Later, a long-ignored **slow drifter wins**: once repaired, it stays on track much longer.

The index weighs urgency against how long a fix will last.

Back to the government: setting the subsidy



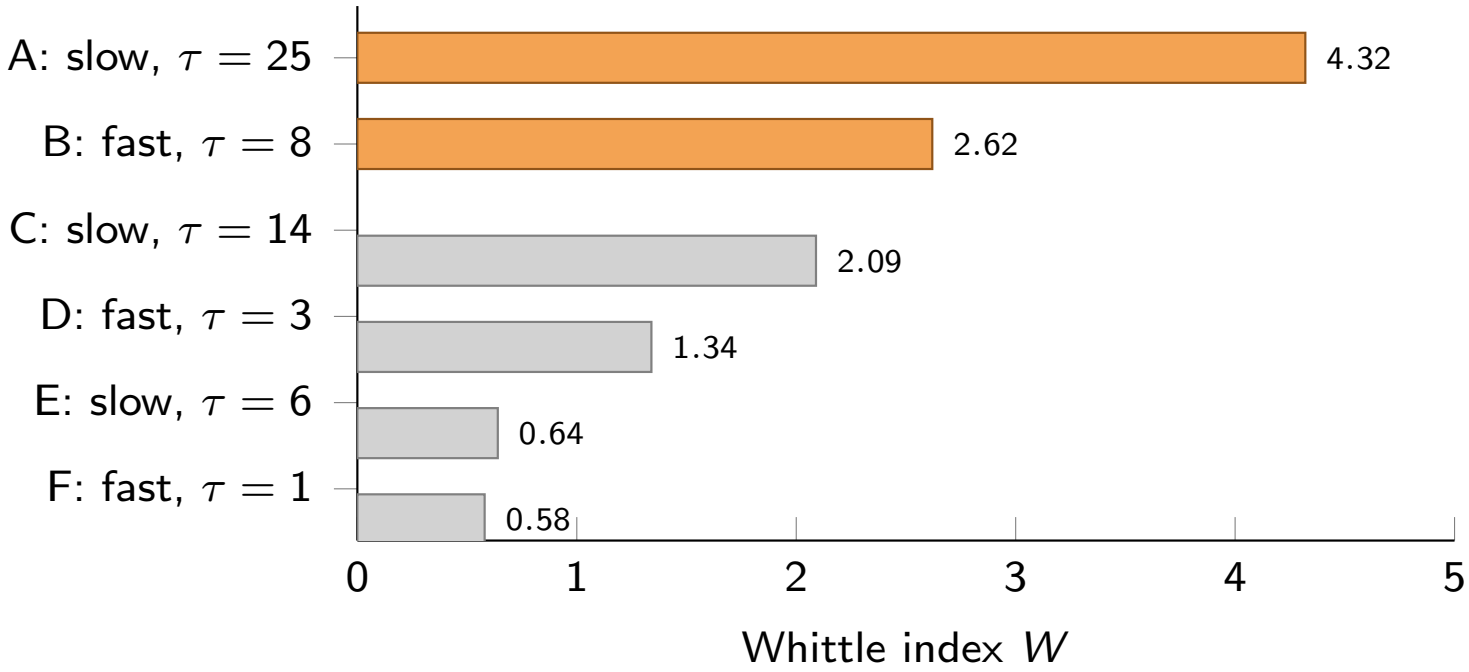
Each field's **break-even subsidy** = its Whittle index.

Fields above the line farm; the rest take λ.

Lower λ: more farm. Raise λ: fewer farm.

Choosing λ so that exactly k farm = farming the k largest indices = the Whittle policy.

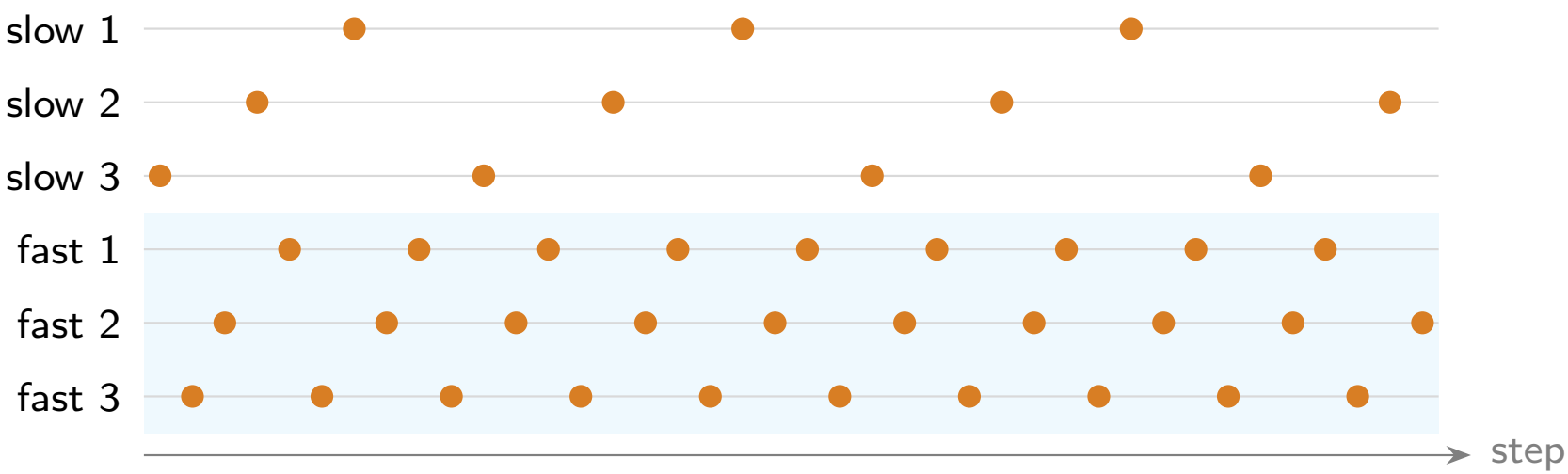
The Whittle policy: rank and check the top k



$k = 2$: check **A** and **B**. The long-ignored slow chain beats the urgent fast one D.

Not optimal in general, but asymptotically optimal as $N \to \infty$ under conditions (Weber & Weiss, 1990).

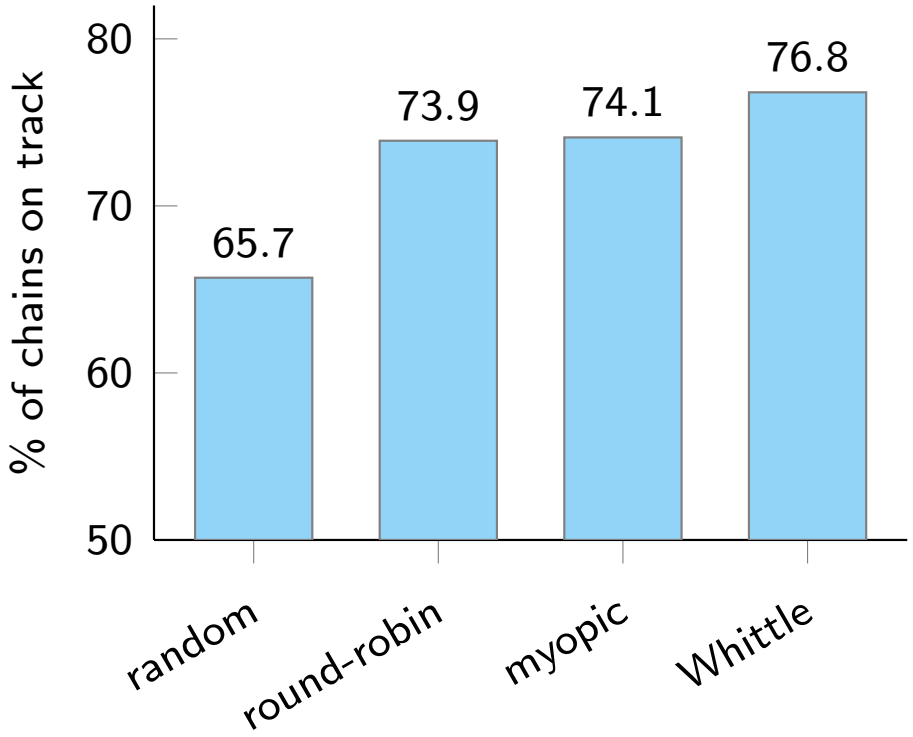
Who gets checked when?



One check per step, Whittle policy. Fast chains: about 10 checks each; slow chains: 3–4, each only once it has been ignored long enough.

The index sets a rhythm: often for fast drifters, rarely for slow ones.

Does it help? A toy simulation



12 chains (6 slow, 6 fast), 2 checks per step, 20,000 steps, 5 seeds.

- *myopic*: check the chains most likely off track now.
- The big jump is from random to any sensible schedule.
- Whittle's extra gain: it also values how long a fix lasts.

Learning when the dynamics are unknown

- In practice p_i (how fast each chain drifts) is unknown.
- Estimate the transitions from the chains we check, plug the estimates into the index, and keep some exploration so every chain gets checked.
- Harder than Lecture 8: a chain's state is seen only when it is checked, and leaving it alone changes it.

Restless bandits in the real world

Deployed to decide which beneficiaries of a maternal-health programme receive a call each week (Mate et al., 2022): limited calls, and engagement drifts whether or not someone is called.

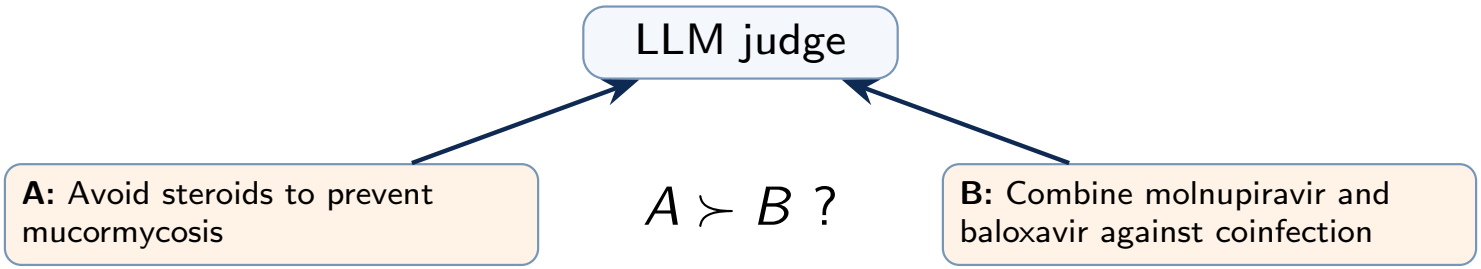
Contextual bandits: the best arm depends on the question

Each round a context x_t arrives (the question); the reward depends on (x_t, a).

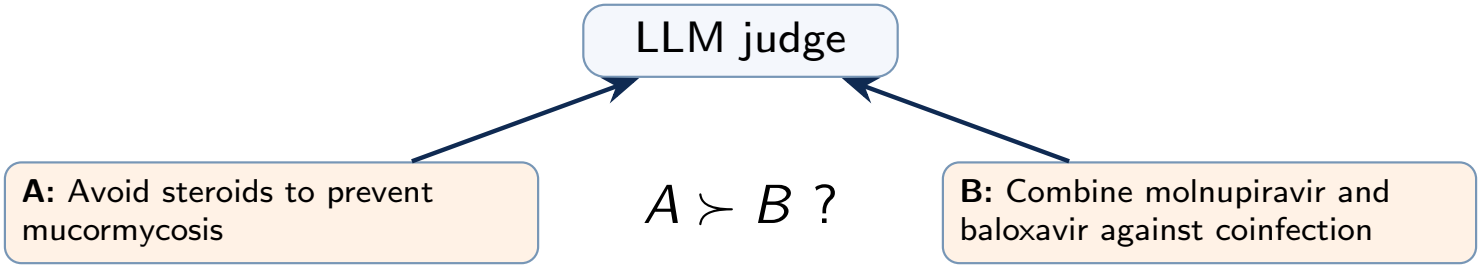
- **Example:** route each question to an exemplar set, a model, or a strategy. EXPLORA picks *one* exemplar set for a task; a contextual learner picks one *per question*.
- **Linear version:** features $\phi(x_t, a)$, reward $\langle \phi(x_t, a), \theta_* \rangle$. This is OFUL / LinUCB with action set $\mathcal{A}_t = \{\phi(x_t, a)\}_a$ (Li et al., 2010).

Sleeping bandits are the special case where the context only switches arms on or off.

Dueling bandits: "A is better than B"

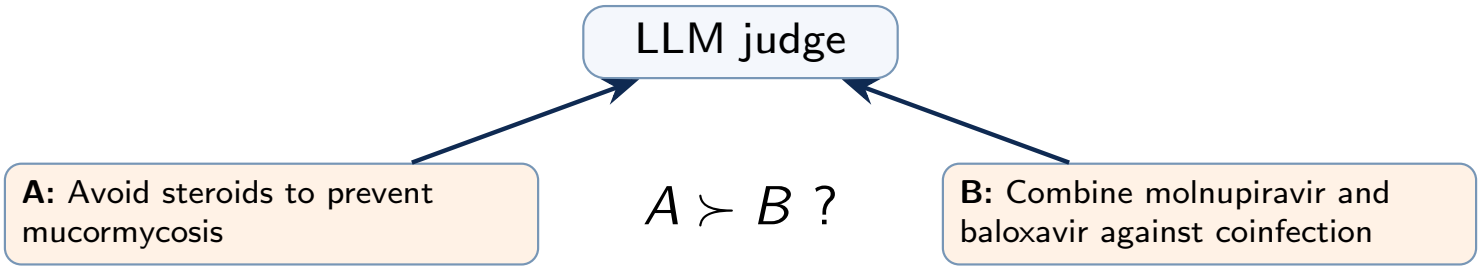


Dueling bandits: "A is better than B"



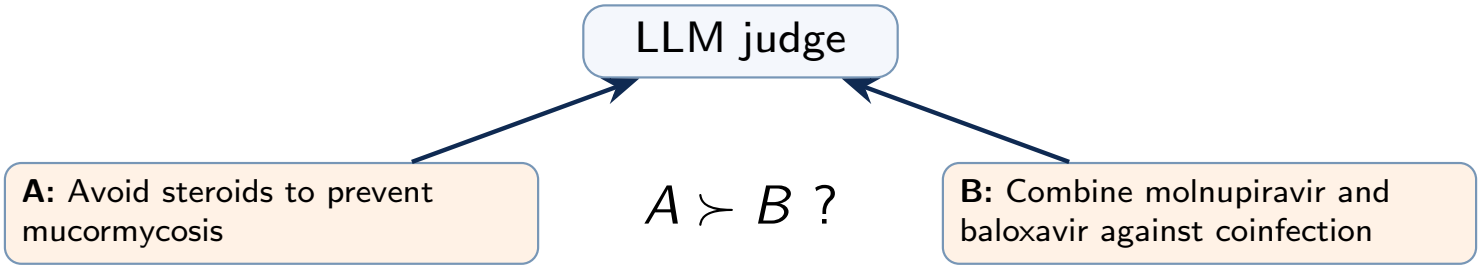
- A pull = compare two arms; feedback is only who won.

Dueling bandits: "A is better than B"



- A pull = compare two arms; feedback is only who won.
- Natural for LLM judges and human preferences (as in RLHF): "which answer is better" is easier than "score this answer".

Dueling bandits: "A is better than B"



- A pull = compare two arms; feedback is only who won.
- Natural for LLM judges and human preferences (as in RLHF): "which answer is better" is easier than "score this answer".
- Goal: find the **Condorcet winner**, the arm that beats every other with probability $> \frac{1}{2}$ (Yue et al., 2012).

Takeaways

- Each assumption of the classic bandit, when broken, gives a family with its own benchmark.
- **Sleeping:** compare only with what was available; UCB over the awake arms; OFUL handles changing action sets for free.
- **Restless:** doing nothing is a decision; the Whittle index turns one coupled problem into N small ones by pricing passivity.
- Test-time scaling needs several at once: sleeping candidates, restless chains, pairwise judges, compute budgets.

Before choosing an algorithm, ask: which assumption does my problem break?

References

- R. Kleinberg, A. Niculescu-Mizil, Y. Sharma. Regret bounds for sleeping experts and bandits. *Machine Learning*, 2010.
- V. Kanade, T. Steinke. Learning hurdles for sleeping experts. *ACM Trans. Computation Theory*, 2014.
- P. Whittle. Restless bandits: activity allocation in a changing world. *J. Applied Probability*, 1988.
- J. C. Gittins. Bandit processes and dynamic allocation indices. *J. Royal Statistical Society B*, 1979.
- C. H. Papadimitriou, J. N. Tsitsiklis. The complexity of optimal queuing network control. *Math. of Operations Research*, 1999.
- R. R. Weber, G. Weiss. On an index policy for restless bandits. *J. Applied Probability*, 1990.
- A. Mate et al. Field study in deploying restless multi-armed bandits. *AAAI*, 2022.
- L. Li et al. A contextual-bandit approach to personalized news article recommendation. *WWW*, 2010.
- Y. Yue et al. The K-armed dueling bandits problem. *J. Computer and System Sciences*, 2012. M. Zoghi et al. RUCB. *ICML*, 2014.
- W. Chen, Y. Wang, Y. Yuan. Combinatorial multi-armed bandit. *ICML*, 2013.
- A. Garivier, E. Moulines. On upper-confidence bound policies for switching bandit problems. *ALT*, 2011.
- P. Auer et al. The nonstochastic multiarmed bandit problem. *SIAM J. Computing*, 2002.
- A. Badanidiyuru, R. Kleinberg, A. Slivkins. Bandits with knapsacks. *FOCS*, 2013.